

school B slightly outperforming school A. But what does the standard deviation value tell us? To further your understanding, I will give you the two lists, 'C1 = [20, 22, 20, 22, 22, 20]' and 'C2 = [18, 16, 17, 16, 15, 48]'. Though hidden by the mean, the large standard deviation value of 11.813363431112899 clearly captures the mass failure in school B. This example clearly tells us that we need more complicated parameters to understand the complex nature of the problems we deal with. Probability and statistics will come to our rescue here by providing more and more complex models which can imitate complex and chaotic data.

Random number generation is an essential part of probability. But, in practice, only pseudorandom number generation (a sequence of numbers whose properties approximate the properties of sequences of random numbers) is possible. Now, let us see a few functions that will help us generate pseudorandom numbers. Consider the code shown in Figure 6. First, let us try to understand the code. Line 1 imports the random package of Python. In Line 2, the function `random.random()` generates random numbers. This line of code, '`new_list = [random.random() for i in range(2)]`', uses a technique called list comprehension to generate two random numbers and store them in the list named `new_list`. Line 3 prints this list as output. Notice that the two random numbers printed change with each iteration of the code, and the probability of getting the same numbers printed twice consecutively is theoretically zero. The single line of code in the second cell shown in Figure 6 uses the function `random.choice()`. This function makes an equally likely choice from all the choices given to it. In this case, the code fragment `random.choice(["Heads", "Tails"])` will make a choice between 'Heads' or 'Tails' with equal probability. Notice that this line of code also uses the technique called list comprehension so that three successive choices between 'Heads' or 'Tails' are made. Figure 6

```
1 import random
2 new_list = [random.random() for i in range(2)]
3 print(new_list)

[0.18869669339036488, 0.4894201070245773]

1 print([random.choice(["Heads", "Tails"]) for i in range(3)])
['Tails', 'Tails', 'Tails']
```

Figure 6: Pseudo random number generation

```
1 import random
2 n=1000
3 ct=0
4 for i in range(n):
5     if random.randint(1, 6) == 6:
6         ct=ct+1
7     print(ct/n)
8     print(1/6)

0.179
0.16666666666666666
```

Figure 7: Illustrating the law of large numbers

shows the output of this code, where the option 'Tails' is chosen for three consecutive times.

Now, let us try to illustrate a very popular theorem in probability, the law of large numbers (LLN), with a simple example. LLN states that the average of the results obtained from a large number of trials should be close to the expected value. Further, this average tends to come closer and closer to the expected value as more and more trials are performed. We all know that if a fair dice is thrown, the probability of getting the number 6 is 1/6. We now simulate this experiment with a simple Python code shown in Figure 7. Line 1 imports the `random` package of Python. Line 2 sets the number of trials to be performed. In this case, it is 1000. Line 3 initialises the counter `ct` to zero. Line 4 sets a loop, which iterates 1000 times in this case. Line 5 uses the function `random.randint(1, 6)`. In this case, the function generates an integer between 1 and 6 (inclusive of both 1 and 6) randomly. The `if` statement in Line 5 checks whether the number generated is equal to 6; if yes the control goes to Line 7 and increases the counter `ct` by 1.

After the loop is iterated a 1000 times, Line 8 will be executed. Line 8 prints the ratio between the number of occurrences of the number 6 and the total number of trials (1000).

Figure 7 also shows this number as 0.179 (slightly more than the expected value $1/6 = 0.1666\dots$ which is also printed in the output). Not that close to the expected value, right? In Line 2, set the value of `n` as 10000. Run the code again and observe the new output printed. Most probably you will get a number which is a little bit closer to the expected value. Notice that this could also be a number less than the expected value. Increase the value of `n`, in Line 2, for a few more times. You will observe that the output is inching closer and closer to the expected value. Thus, with a simple code, we have illustrated LLN.

Though a simple statement, you will be amazed to know that the list of mathematicians who worked on a proof for LLN or tried to refine one include Cardano, Jacob Bernoulli, Daniel Bernoulli, Poisson, Chebyshev, Markov, Borel, Cantelli, Kolmogorov, Khinchin, etc. All of them are mathematical giants in their own field.

We are yet to cover topics in probability like random variables, probability distributions, etc, which are essential in developing AI and machine learning based applications. Our discussion of topics in probability and statistics is still in the early stages and we will try to strengthen our knowledge of these subjects in the next article also. At the same time, we will revisit two of our old friends we met in this journey earlier, the libraries Pandas and TensorFlow. We will also befriend a relative of TensorFlow called Keras, a library which acts as an interface for TensorFlow. **END** 🐼

 By: Deepu Benson

The author is a free software enthusiast. His area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. He maintains his own technical blog too.